Operating System & Advanced Mobile Lab 2

Report – Multi-threaded word count

By Sebastien BERTIL SOUCHET (32239360)

Date: 2025/10/19

1. Introduction

This project implements a **multi-threaded word/character count** in C using POSIX threads.
It focuses on building a correct and efficient **producer–consumers** pipeline to process large text files.

Goals

- Read an input text file line by line (producer).
- Safely share data through a **bounded circular queue** protected by mutex and condition variables.
- Run **N consumers** in parallel to process lines and **count A..Z occurrences** (case-insensitive).
- Aggregate per-thread statistics at the end and print global totals.
- Expose simple CLI controls: input file, number of consumers, buffer capacity.
- Measure elapsed time to compare throughput across thread counts.

What it supports

- 1 producer (I/O) + configurable **N consumers** (CPU).
- Executing Unix-like systems with **pthreads**.
- Robust synchronization (no data races), **graceful shutdown** on EOF (broadcast wakeup).
- Minimal error handling (file open errors, allocation failures, invalid args).

2. Instructions to Build the Program

Requirements

- GCC (or Clang)
- Linux / Unix-like environment
- POSIX threads (pthreads) available by default on Linux

Files

- **main.c**: program entry, argument parsing, thread orchestration (producer + N consumers), timing, final aggregation.
- **consumer.c**: consumer thread routine and local A..Z counters.
- **queue_tools.c**: bounded circular queue + synchronization (mutex, not_empty, not_full, done flag).
- **prod_cons.h**: shared types, constants, and function prototypes.

- **Makefile**: build automation.

Build

Run:

make

This produces the binary (per your repo):

./word_count

To clean build artifacts:

make clean

To clean everything (artifacts & binaries):

make fclean

To rebuild from scratch:

make re

Example run

./word_count ./Electric_Vehicle_Population_Data.csv 1 4 4096

3. How the Program Works

General Flow

1. **Args parsing**
   The program reads CLI arguments: <file> <#producers> <#consumers> [buffer_capacity].
   It forces the effective number of producers to **1** (as required).
2. **File open & queue init**
   a. Opens the input file in binary/text mode.

   b. Initializes a **bounded circular queue** (cap entries) protected by a mutex and two condition variables:
      i. not_empty (consumers wait when the queue is empty)
      ii. not_full (producer waits when the queue is full)
3. **Thread start**
   a. **Producer thread**: starts reading lines with getline.
   b. **N consumer threads**: start waiting to pop work from the queue.
4. **Producer loop (I/O stage)**
   a. For each line: getline → strdup → queue_push (blocks if full).
   b. On EOF: sets done = true then pthread_cond_broadcast(not_empty) to wake all consumers.
5. **Consumer loop (CPU stage)**
   a. queue_pop (blocks if empty and done == false).
   b. When a line is obtained: scan characters, convert to upper (toupper) and update the **local A..Z counters** (no locks needed on this hot path).
   c. free(line) and repeat.
   d. Exit when done == true**and** the queue is empty.
6. **Join & aggregation**
   a. main joins all consumers, then the producer.
   b. Sums each consumer's local A..Z table into a **global** result.
7. **Report**
   a. Prints a short summary (producers/consumers/buffer, lines produced/consumed, elapsed time).
   b. Prints the **A..Z totals** in tabular form.

Execution Paths & Decisions

- **Queue full** → producer waits on not_full.
- **Queue empty** → consumers wait on not_empty.
- **EOF reached** → done=true + broadcast to allow consumers to exit cleanly once the queue drains.
- **Errors** (file open, thread create, memory) → message on stderr and exit.

Why it's correct & fast (in brief)

- **Correctness**: All shared queue state is guarded by a mutex; consumers exit only when done=1 **and** the queue is empty, avoiding races and hangs.
- **Performance**: Counting is done **per-thread locally** (no shared counters in the hot loop), so there's no contention while scanning characters. Aggregation happens once at the end.

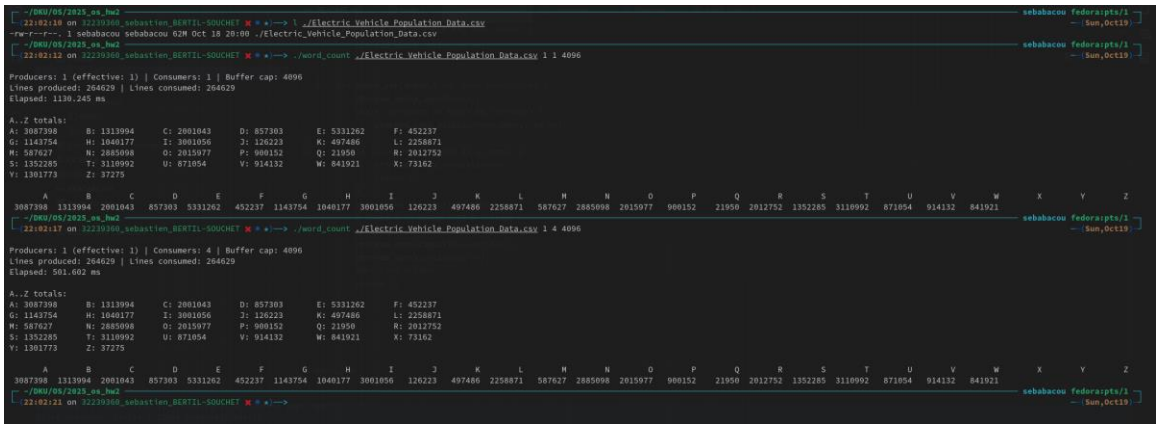
4. Example Usage

```
./word_count ./Electric_Vehicle_Population_Data.csv 1 4 4096

Producers: 1 (effective: 1) | Consumers: 4 | Buffer cap: 4096
Lines produced: 264629 | Lines consumed: 264629
Elapsed: 489.006 ms

A..Z totals:
A: 3087398      B: 1313994      C: 2001043      D: 857303       E: 5331262      F: 452237
G: 1143754      H: 1040177      I: 3001056      J: 126223       K: 497486       L: 2258871
M: 587627       N: 2885098      O: 2015977      P: 900152       Q: 21950        R: 2012752
S: 1352285      T: 3110992      U: 871054       V: 914132       W: 841921       X: 73162
Y: 1301773      Z: 37275
```

5. Sample Screenshot (terminal capture)



6. Usage of AI

I used an AI assistant to help refine the **display function** print_stats so the final output is clearer and more readable (aligned columns, consistent spacing, grouped A..Z totals).

Concretely, the AI helped me:

- Choose a **portable printf format** for 64-bit counters using PRIu64 (e.g., printf("%" PRIu64, value);).
- Improve the **tabular layout** (line breaks, column alignment, compact grouping).

Scope & attribution:

- The AI was used **only** for presentation/formatting of the output and minor portability tips.

- All **core logic** (producer/consumers, queue synchronization, counting and aggregation) was designed and implemented by me, and I reviewed/validated every change before integrating it.